

JUSTIFICACIÓN DEL USO DE PATRONES DE DISEÑO

En el desarrollo del sistema de gestión de eventos y venta de entradas se seleccionaron distintos patrones de diseño con el objetivo de resolver problemas específicos de arquitectura, mejorar la escalabilidad del sistema y evitar soluciones rígidas basadas únicamente en condicionales o herencia. A continuación, se justifica su uso frente a otras alternativas posibles.

Singleton

Se utilizó Singleton para garantizar una única instancia global del sistema.

Se eligió este patrón en lugar de crear múltiples instancias de clases de control o usar variables globales, ya que estas últimas pueden generar inconsistencias en los datos y dificultar el control del estado del sistema. Tampoco se consideró necesario el uso de frameworks más complejos de inyección de dependencias debido al alcance del proyecto.

Factory Method

Se utilizó Factory Method para la creación de eventos.

Se eligió en lugar de la instanciación directa con `new`, ya que esta alternativa acopla el código a clases concretas y dificulta la extensión del sistema. También se evitó el uso de constructores múltiples o lógica condicional (`if-else`) para decidir el tipo de evento, ya que esto aumenta la complejidad y reduce la mantenibilidad.

Prototype

Se utilizó Prototype para la creación de objetos mediante clonación.

Se eligió en lugar de la creación manual de objetos o el uso de constructores extensos, ya que estos generan repetición de código cuando los objetos comparten configuraciones similares. Tampoco se utilizó Builder en la versión final, ya que su complejidad no era necesaria para el nivel de variación de los objetos del sistema.

Composite

Se utilizó Composite para representar la relación entre zonas y asientos.

Se eligió en lugar de estructuras basadas en listas simples o relaciones planas, ya que estas no permiten modelar jerarquías parte-todo de forma clara. Tampoco se utilizó una estructura basada únicamente en agregaciones simples, ya que no permite tratar objetos compuestos y simples de forma uniforme.

Decorator

Se utilizó Decorator para la gestión de servicios adicionales en compras.

Se eligió en lugar de la herencia, ya que esta última generaría una gran cantidad de subclases para cada combinación posible de servicios (VIP, parqueadero, merchandising). También se evitó el uso de condicionales dentro de la clase Compra, ya que esto haría el código más rígido y difícil de extender.

Adapter

Se utilizó Adapter para integrar una librería externa de reportes.

Se eligió en lugar de modificar directamente la librería externa, ya que esto no es posible ni recomendable. Tampoco se optó por acoplar directamente el sistema a la API, ya que esto generaría dependencias fuertes y falta de flexibilidad ante cambios.

Strategy

Se utilizó Strategy para los métodos de pago.

Se eligió en lugar de estructuras condicionales como `switch` o `if-else`, ya que estas aumentan la complejidad cuando crecen los métodos de pago. Tampoco se utilizó herencia directa, ya que esta no permite cambiar el comportamiento dinámicamente en tiempo de ejecución.

Proxy

Se utilizó Proxy para controlar el acceso a las operaciones de compra.

Se eligió en lugar de incluir validaciones directamente en la clase Compra, ya que esto violaría el principio de responsabilidad única. También se evitó delegar estas validaciones en controladores o servicios principales, ya que esto generaría código disperso y difícil de mantener.

Observer

Se utilizó Observer para la notificación de eventos e incidencias.

Se eligió en lugar de llamadas directas entre clases, ya que esto generaría alto acoplamiento. Tampoco se utilizó polling o verificaciones constantes del estado, ya que esto sería ineficiente y poco escalable.

State

Se utilizó State para la gestión de estados de compras y asientos.

Se eligió en lugar de condicionales basadas en estados (`if-else` o `switch`), ya que estas vuelven el código difícil de mantener y propenso a errores. Tampoco se utilizó enums con lógica interna compleja, ya que esto mezcla responsabilidades dentro de una sola estructura.

CONCLUSIÓN

Los patrones de diseño seleccionados permiten resolver problemas específicos del sistema de manera modular y escalable. Cada patrón fue elegido frente a alternativas más simples (como condicionales, herencia directa o acoplamiento fuerte), priorizando mantenibilidad, flexibilidad y separación de responsabilidades.